

Code Quality

Course Goal

Build software

Build software

With other people

Build software

With other people

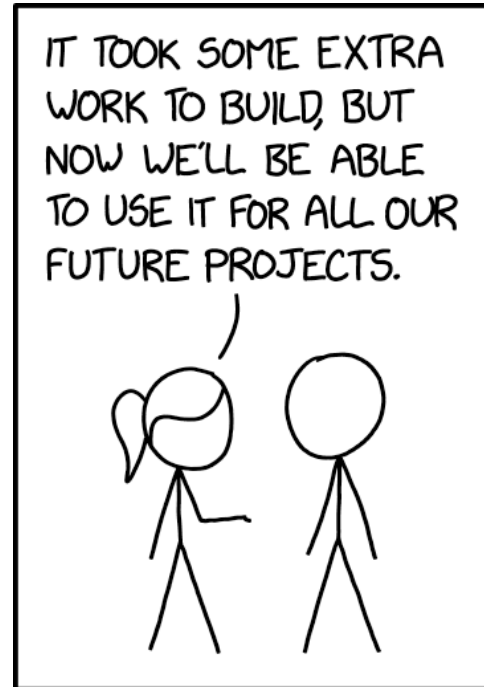
Effectively

Build software

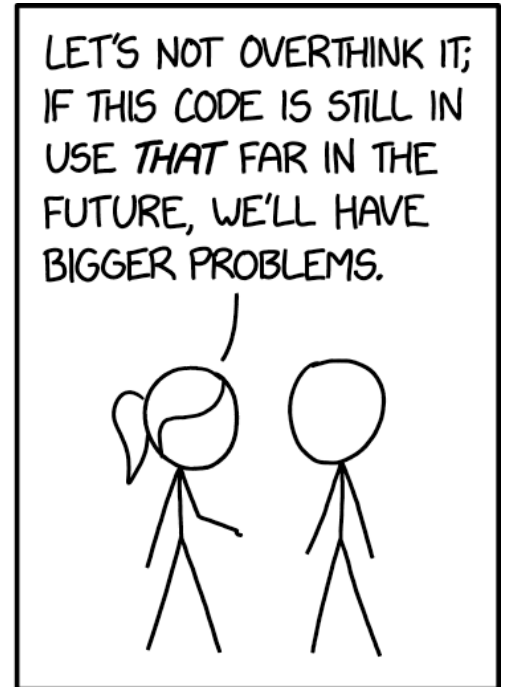
With other people

Effectively

Over time



HOW TO ENSURE YOUR CODE IS NEVER REUSED



HOW TO ENSURE YOUR CODE LIVES FOREVER

Course Philosophy

1. Build a useful foundation.

- Learn the theory of version control, testing, dependency management, code quality, and design patterns.

2. Practice with real tools.

- Use Git, Gradle, Java, JUnit and other tools to build software.

Development Ecosystem

Version Control + Local Checkout + Build Tool + IDE + Testing Framework + CI/CD

Git + Gradle + Eclipse/IntelliJ/VS Code + JUnit + GitHub Actions

Supported vs. Allowed

- **Supported:** Tools, practices, and standards officially endorsed and recommended by the organization or project.
- **Allowed:** Tools, practices, and standards that are permitted but not officially recommended. No guarantees of support!

Specific Tools

Git/GitHub = required

Gradle = required

Eclipse = recommended

IntelliJ = allowed

VS Code = allowed

JUnit = required

Java = required!!!

Setup your IDE!

- Gather your tools and begin to set them up before the next lecture.
- We will go into more detail about all the tools in the coming lectures.
- You should be able to write/compile/run Java code.

What is Code Quality?

Code quality refers to the characteristics of code that make it reliable, maintainable, and easy to understand. High-quality code is:

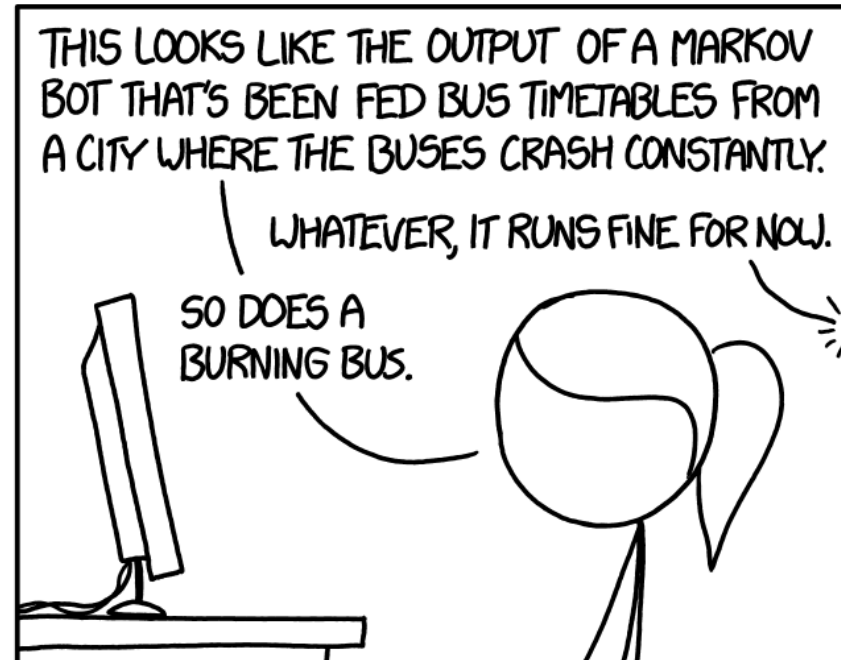
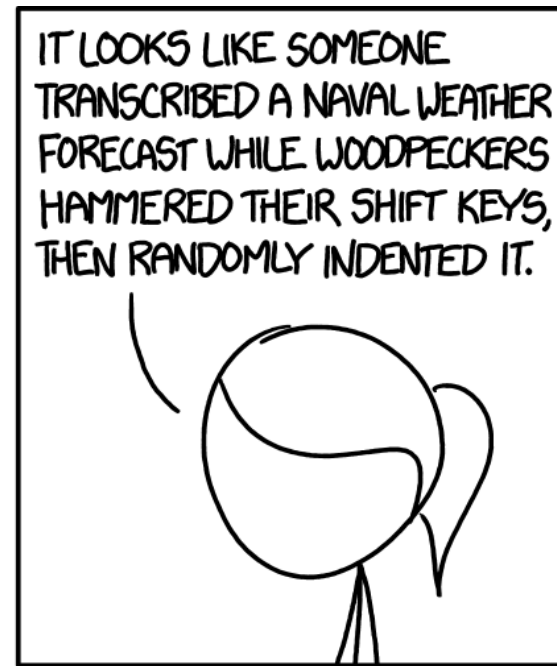
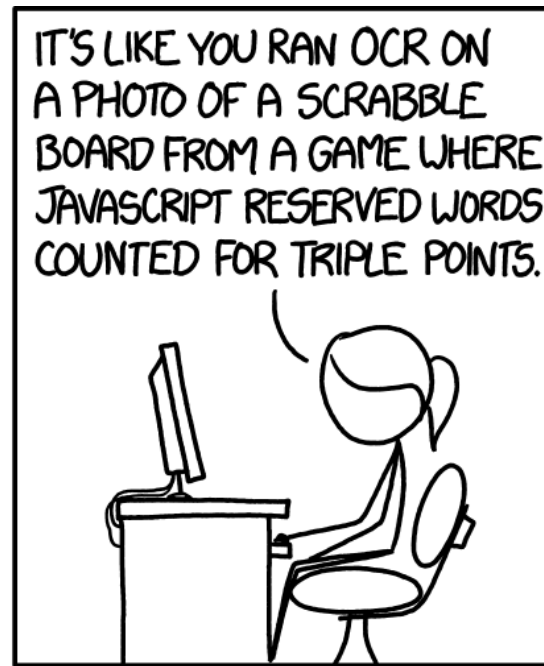
- **Readable:** Easy for other developers to understand.
- **Maintainable:** Easy to modify and extend.
- **Reliable:** Functions correctly and is robust to errors.
- **Efficient:** Uses resources wisely.

Code Review

Your code will be read by other developers, your future self, and more LLMs.

Best practices make the process:

- **Lightweight:** Easy to review and understand.
- **Effective:** Catches issues early and improves code quality.
- **Pleasant:** Encourages collaboration and knowledge sharing.



Structure your code

Style **conventions** are aspects of code not enforced by the compiler but are important for readability and maintainability.

Clear and understandable:

```
int getWidth() {  
    return width;  
}  
  
rectangle.getWidth();
```

Unclear and confusing:

```
int Width(){retun a};  
  
b.Width();
```

Your code should be readable and understandable by other developers, including your future self.

Structure your code

Typical style conventions include:

- Methods and variables begin with a lowercase letter. (e.g., `getWidth()` , `width`)
- Constants are in all uppercase letters with underscores separating words. (e.g., `MAX_WIDTH`)
- Classes and interfaces begin with an uppercase letter. (e.g., `Rectangle` , `TestCase`)

Optional style conventions:

- Indentation: tabs vs 2 spaces vs 4 spaces
- `{` on the same line vs on a new line

Structure your code

Commenting your code is important for readability and maintainability. Comments should explain **why** something is done, not **what** is done. I can look at the code to see what it does. Your implementation could have been done in many different ways, but you chose this way. Why?

Don't just explain **what**:

```
// Checks if password length is greater than or equal to 12
if (password.length() >= 12) {
    savePassword(password);
}
```

Explain **why**:

```
// NIST 800-63B guidelines require a minimum of 12 characters to prevent dictionary attacks
if (password.length() >= 12) {
    savePassword(password);
}
```

Structure your code

Remember good **object oriented design** principles:

- A **class** should have a single responsibility/concept.
- A **method** should have one task/behavior.

If you can't explain what a class or method does in one sentence, it probably has too many responsibilities and should be split into multiple classes or methods.

```
getWidthAndHeight()
```

```
getWidth()  
getHeight()
```

Structure your code

Every rule has exceptions.

Sometimes it is better to break the rules for clarity and maintainability.

```
//Represents a deck of cards. Supports shuffling and dealing.  
  
public class Deck {  
    List<Card> cards;  
    public void shuffle() {}  
    public Card deal() {}  
}
```

There are multiple sentences and an 'and', but there's a single 'data' concept and the two 'behaviors' are both closely related to the data and in separate methods.

Structure your code

Bad examples:

```
// Represents card numbers and suits.  
public class Deck {  
    List<Integer> cardNumbers;  
    List<String> cardSuits;  
}
```

The "and" here is a red flag. Managing raw parallel lists creates fragile data structures. Cards should be **encapsulated** into their own class.

```
/* Represents a deck of cards. Supports shuffling and coordinates placing pieces on a game board. */  
public class Deck {  
    List<Card> cards;  
    GameBoard board;  
  
    public void shuffle() {}  
    public void placePieceOnBoard(Piece piece) {}  
}
```

Here, the "and" is a true red flag. The two behaviors belong to entirely different domains. A deck of cards should not handle game board placement logic.

Structure your code

Code should not be surprising:

Main.java

```
System.out.println("Rectangle width: " + rectangle.getWidth());
```

Rectangle.java

```
public int getWidth() {  
    setHieght(3); // This is surprising! A getter should not have side effects.  
    return width;  
}
```

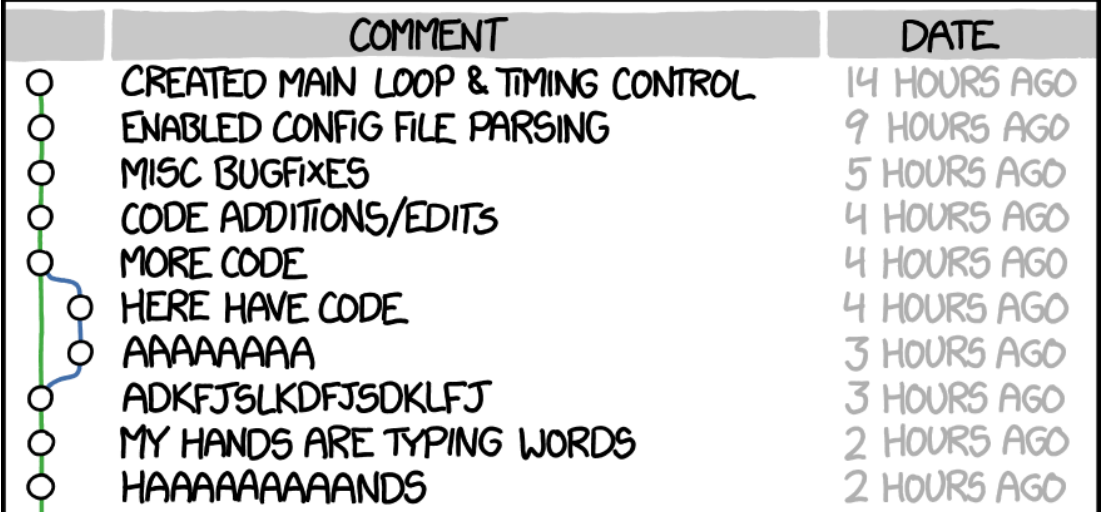
You should not be surprised by what a method does. If a method has side effects, it should be clearly documented and named appropriately.

Structure your code

Manage your commit messages and history.

- Each commit should represent a single logical change.
- Avoid large commits that mix multiple changes together.
- Less than 200 lines of non-auto-generated code is a good rule of thumb for a single commit.
- A single feature/bug/task!
- Detailed description of the change.

We will go into more detail about Git and commit messages in the coming lectures.



	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFT	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

Reviewing code

Step 1: Grok the code (Understand it

- Read the commit summary and description.
- Skim through the whole change.
- Do a second pass for a deeper understanding.
 - Make notes of anything that looks unclear
 - If necessary, sketch out a diagram/outline of what the change is doing.

Reviewing code

Step 2: Comment on largers issues

BE NICE!!! Criticism always feels harsher on the receiving end. Be constructive and polite.

A code reivew is a conversation, not a judgement.

Topics to discuss:

- Bugs: if something looks funcationally broken
- Lack of testing: make sure there are at least basic tests
- Unclear code: missing comments, code that isn't self-documenting, confusing variable/method names
- Structural issues: layers of abstraction or API design.

Reviewing code

Step 3: Proofreading for stylistic issues

For small issues: by convention, start with "nit:" E.g., "nit: rename variable to be more descriptive". Yes "Nit" is short for "nitpick".

- Whitespace/formatting issues
- Naming issues: variable, method, class names
- Suboptimal return statements
 - E.g., "nit: return hasProperty() vs return hasProperty == true"

These shouldn't block approvals, but should be fixed before merging. If you see a lot of these, it may indicate a larger issue with the codebase or the developer's understanding of the code.

Avoid micromanaging!

- Determine if another way is genuinely better, or just different.

Formatting

Code formatters automatically apply a consistent style to your code. This eliminates debates over style and improves readability. It's suboptimal to spend a lot of time on nits.

Fortunately, some of these issues can be automatically fixed with code formatters.

A **linter** that cleans up the "lint" in your code. The small bits of easily removed fluff that can be cleaned up automatically.

Linting

Linters perform static analysis on your code to find potential issues, bugs, and stylistic errors without running it.

- **Checkstyle for Java:** A popular linter for Java that enforces a coding standard. We will use it in this course to enforce a consistent style and catch common issues.
- **Linter Rules:** Linters have a set of configurable rules. You can disable rules for specific lines or files if needed.

Checkstyle with Github

We are going to add checkstyle to our GitHub (to be continued in the next lecture).

Every Pull Request will automatically trigger these checks:

- Don't request a review until all checks pass.
- They should be very quick to run and fix!

This is an example of Continuous Integration (CI) in action. We will discuss CI/CD in more detail later in the course.

Note: There are a lot of ways to use checkstyle. In the Gradle and in you IDE. Feel free to add additional checks to you project!

Continuous Integration (CI)

- Shorten the loop between code being written and being deployed to production.
- Automate error detection, shipping, and rollbacks
- Complexity ranges from very simple (checkstyle) to very complex (running a full test suite, building and deploying a Docker image, etc.)

Benefits:

- Faster feedback
- Easier to launch new features
- Higher quality code

Pair Programming

Combine the coding and code review steps.

Requires two developers working at the same time on the same screen or on a screenshare (e.g., Discord, Zoom, Google Meet, etc.) Email does not count!

One person types, the other reviews.

Good for the first several commits of a large, complex architecture

- Avoids latency and context switching from code reviews

When committing, note in the **commit message** that the code was pair programmed (and with who). This should line up with the person who **approves** the pull request.

Where to write code

Integrated **D**evelopment **E**nvironments (IDEs) are software applications that provide comprehensive facilities to programmers for software development.

- Constant compilation and error checking
- Syntax highlighting and auto-completion
- Automated refactoring and code generation
- Debugging tools

IDE for Java

- **Eclipse:** Another widely used IDE for Java with extensive tooling support. (Recommended for this course)
- **IntelliJ IDEA:** A popular IDE for Java development with powerful features and plugins. (Allowed)
- **VS Code:** A lightweight, extensible code editor that can be configured for Java development with extensions. (Allowed)

Unlock the Power of Your IDEA

A basic text editor is not enough for professional software development. IDEs provide powerful features that can significantly improve your productivity and code quality.

- Solves the complexity to type vs complexity to read. IDEs help you write code that is easier to read and understand.
- Minimizes "toil."
- Allows for easy navigation of object-oriented codebases..

Shortcuts in the IDE are **HUGE** for productivity. Learn them and use them!

Deterministic vs Stochastic Code Generation

Deterministic: Compiler and many of the IDE tools

- No need to double-check output. "Just glance through the bytecode (class files) to make sure it's right" said no one ever.
- Solves specific problems in a specific way. (e.g., auto-formatting, refactoring, code generation)

Non-deterministic: LLMs and other generative AI tools

- Must double-check output. "Just glance through the code to make sure it's right" said everyone.
- Can attempt any problem, but no guarantee of correctness. (e.g., code generation, code completion, code explanation)

For any problem that **can** be solved deterministically, **prefer that option**

This requires **knowing when those options exist**. This is a skill that comes with experience and practice.

Organize Imports

Eclipse shortcut: `ctrl + shift + o` (Windows/Linux) or `cmd + shift + o` (Mac)

Doesn't just "organize" existing imports, it also adds missing imports and removes unused ones. It searches the classpath for the correct classes to import. This is a huge time saver and reduces errors.

Class API design:

- Import package structure is an **implementation detail**.
- The library classes/interfaces are the **API**.

Organize imports = "HEY IDE, I want this class. Go find it for me."

Auto-Refactor

Eclipse shortcut:

- Rename: `Alt + Shift + R` (Windows/Linux) or `Cmd + Alt + R` (Mac)
- Extract Method: `Alt + Shift + M` (Windows/Linux) or `Cmd + Alt + M` (Mac)

Rename Classes/parameters/methods/interfaces

Extract Methods

Reduces the cost of readability/maintainability fixes.

Autocomplete

Eclipse shortcut: `ctrl + space` (Windows/Linux) or `Cmd + Space` (Mac)

Long variables/methods

Prioritize readability over typing effort. Use autocomplete to reduce the cost of long names.

Forgot a method? The IDE is here to help!

Eclipse shortcut: type `.` and wait

Efficiently use a large number of libraries.

Navigate Object Hierarchies

Eclipse shortcut: `ctrl + click` (Windows/Linux) or `cmd + click` (Mac)

Object-oriented programming has tons of classes, interfaces, methods, and variables that exist across multiple files.

Navigating the file structure is very slow!

Context switching is expensive. Use the IDE to navigate the codebase efficiently.

Reasoning about the whole system is difficult. Use the IDE to navigate the codebase efficiently.

Click through navigation in the IDE is much faster than searching through the file structure.

Many more!

Deterministic generation (no generative AI just yet):

- hashCode>equals/toString generation
- getters/setters generation
- try/catch/finally generation
- unimplemented methods generation

Auto-save files

"On save" actions (e.g., auto-format, organize imports, run linter)

And a bunch more!

